Deep Learning (CAI-466)

Objectives:

- R-CNN
- Fast R-CNN
- Faster R-CNN
- Mask R-CNN

R CNN

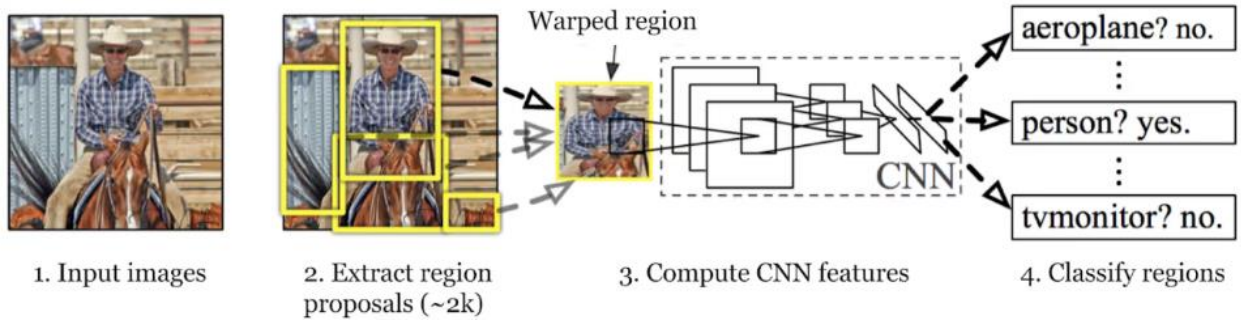
Region-based Convolutional Neural Network (R-CNN) is a type of deep learning architecture used for object detection in computer vision tasks. RCNN was one of the pioneering models that helped advance the object detection field by combining the power of convolutional neural networks and region-based approaches.

In 2014 R-CNN marked a pivotal moment in the field of computer vision. This work not only demonstrated the potential of Convolutional Neural Networks (CNNs) for achieving high performance in object detection but also addressed a challenging problem: the precise localization of objects within images through the creation of bounding boxes.

The R-CNN model's object detection process involves three main steps:

- generating region proposals,
- extracting features,
- and classifying objects while refining their bounding boxes.

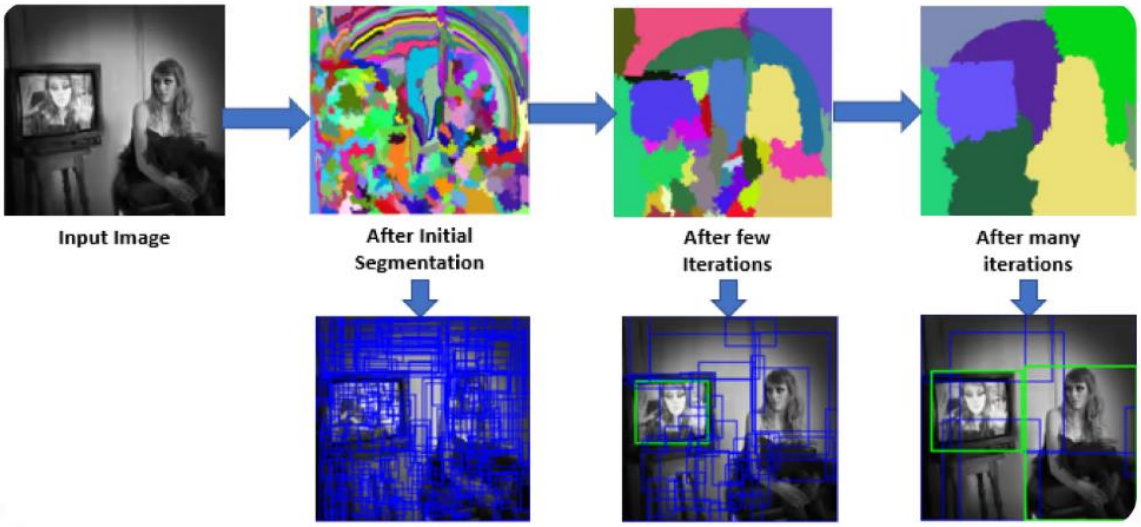
Let's walk through each step.



1) Region proposals:

In the first step, the R-CNN model scans the image to create numerous region proposals. Region proposals are potential areas that might contain objects. Methods like Selective Search are used to look at various aspects of the image, such as color, texture, and shape, breaking it down into different parts. Selective Search starts by dividing the image into smaller parts, then merging similar ones to form larger areas of interest. This process continues until about 2,000 region proposals are generated.

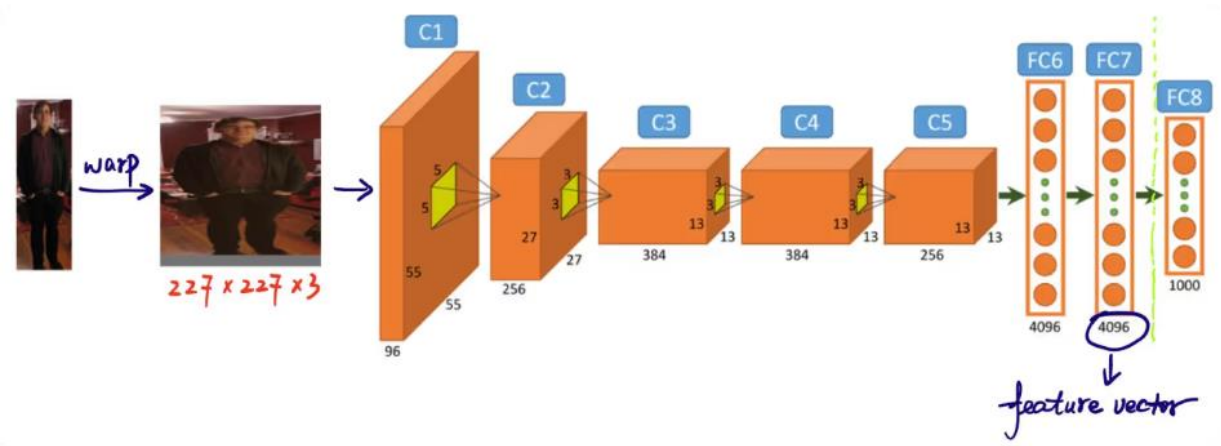
These region proposals help identify all the possible spots where an object might be present. In the following steps, the model can efficiently process the most relevant areas by focusing on these specific areas rather than the entire image. Using region proposals balances thoroughness with computational efficiency.



2) Image feature extraction:

The next step in the R-CNN model's object detection process is to extract features from region proposals. Each region proposal is resized to a consistent size that the CNN expects (for example, 224x224 pixels). Resizing helps the CNN process each proposal efficiently. Before warping, the size of each region proposal is expanded slightly to include 16 pixels of additional context around the region to provide more surrounding information for better feature extraction.

Once resized, these region proposals are fed into a CNN like AlexNet, which is usually pre-trained on a large dataset like ImageNet. The CNN processes each region to extract high-dimensional feature vectors that capture important details such as edges, textures, and patterns. These feature vectors condense the essential information from the regions. They transform the raw image data into a format the model can use for further analysis. Accurately classifying and locating objects in the next stages depends on this crucial conversion of visual information into meaningful data.



3) Object classification: Identifying detected objects

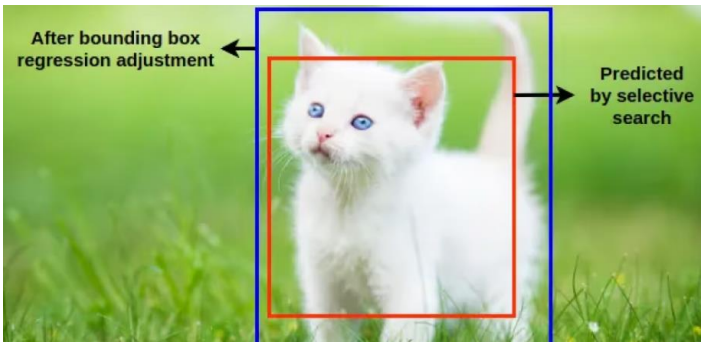
The third step is to classify the objects within these regions. This means determining the category or class of each object found within the proposals. The extracted feature vectors are then passed through a machine learning classifier.

In the case of R-CNN, Support Vector Machines (SVMs) are commonly used for this purpose. Each SVM is trained to recognize a specific object class by analyzing the feature vectors and deciding whether a particular region contains an instance of that class. Essentially, for every object category, there is a dedicated classifier checking each region proposal for that specific object.

During training, the classifiers are given labeled data with positive and negative samples:

- Positive samples: Regions containing the target object.
- Negative samples: Regions without the object.

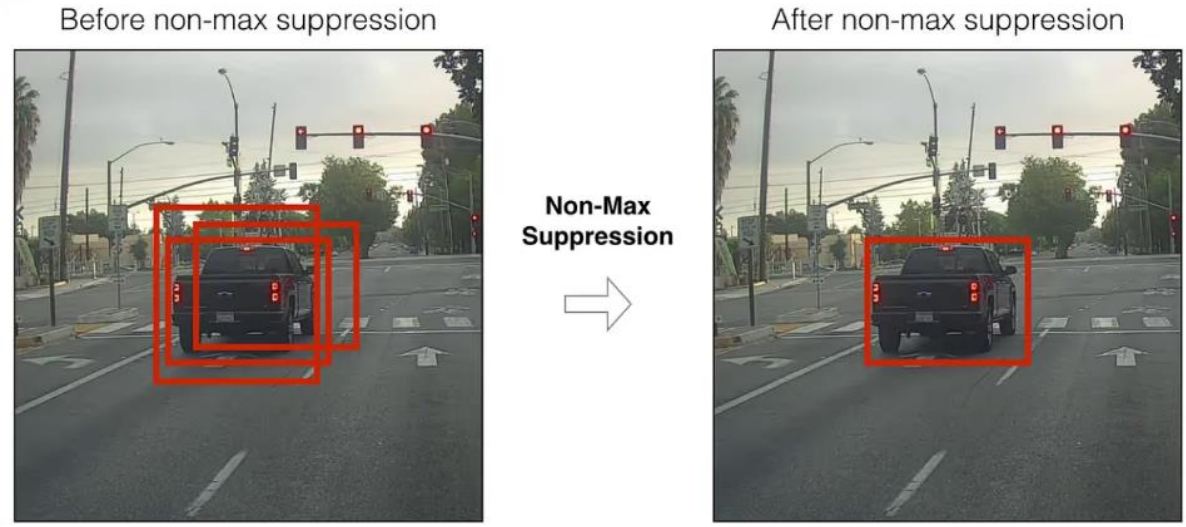
The classifiers learn to distinguish between these samples. Bounding box regression further refines the position and size of detected objects by adjusting the initially proposed bounding boxes to better match the actual object boundaries. The R-CNN model can identify and accurately locate objects by combining classification and bounding box regression.



Putting it all together: Refining detections with NMS

After the classification and bounding box regression steps, the model often generates multiple overlapping bounding boxes for the same object. Non-Maximum Suppression (NMS) is applied

to refine these detections, keeping the most accurate boxes. The model eliminates redundant and overlapping boxes by applying NMS and keeps only the most confident detections.
NMS works by evaluating the confidence scores (indicating how likely a detected object is actually present) of all bounding boxes and suppressing those that significantly overlap with higher-scoring boxes.



To put it all together, The R-CNN model detects objects by generating region proposals, extracting features with a CNN, classifying objects and refining their positions with bounding box regression, and using Non-Maximum Suppression (NMS) keeping only the most accurate detections.

Strengths of R-CNN

Below are a few of the key strengths of the R-CNN architecture.
Accurate Object Detection: R-CNN provides accurate object detection by leveraging region-based convolutional features. It excels in scenarios where precise object localization and recognition are crucial.

Robustness to Object Variations: R-CNN models can handle objects with different sizes, orientations, and scales, making them suitable for real-world scenarios with diverse objects and complex backgrounds.

Flexibility: R-CNN is a versatile framework that can be adapted to various object detection tasks, including instance segmentation and object tracking. By modifying the final layers of the network, you can tailor R-CNN to suit your specific needs.

Disadvantages of R-CNN

Below are a few disadvantages of the R-CNN architecture.

Computational Complexity: R-CNN is computationally intensive. It involves extracting region proposals, applying a CNN to each proposal, and then running the extracted features through a classifier. This multi-stage process can be slow and resource-demanding.

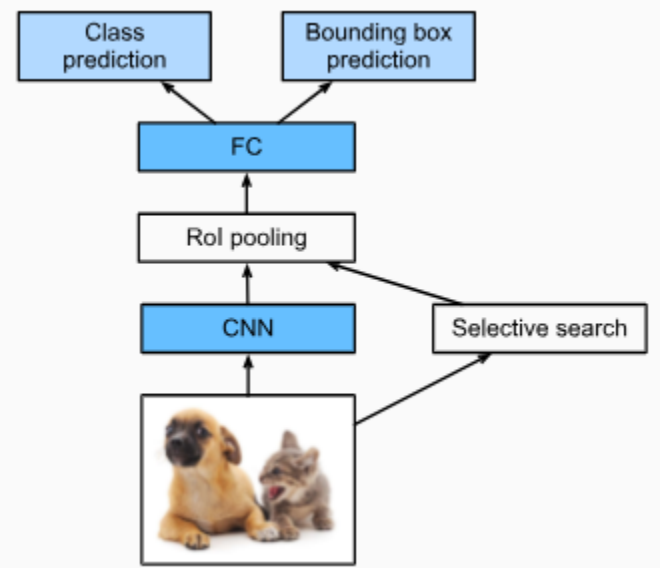
Slow Inference: Due to its sequential processing of region proposals, R-CNN is relatively slow during inference. Real-time applications may find this latency unacceptable.

Overlapping Region Proposals: R-CNN may generate multiple region proposals that overlap significantly, leading to redundant computation and potentially affecting detection performance.

R-CNN is Not End-to-End: Unlike more modern object detection architectures like Faster R-CNN, R-CNN is not an end-to-end model. It involves separate modules for region proposal and classification, which can lead to suboptimal performance compared to models that optimize both tasks jointly.

Fast R-CNN

The main performance bottleneck of an R-CNN lies in the independent CNN forward propagation for each region proposal, without sharing computation. Since these regions usually have overlaps, independent feature extractions lead to much repeated computation. One of the major improvements of the *fast R-CNN* from the R-CNN is that the CNN forward propagation is only performed on the entire image.



Above describes the fast R-CNN model. Its major computations are as follows:

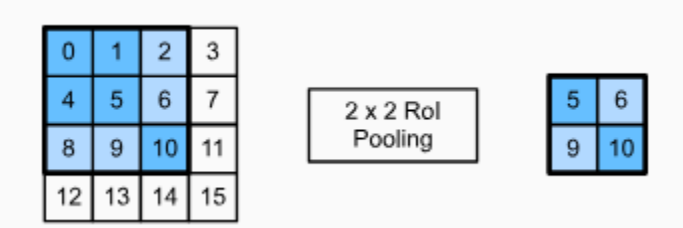
1. Compared with the R-CNN, in the fast R-CNN the input of the CNN for feature extraction is the entire image, rather than individual region proposals. Moreover, this CNN is trainable. Given an input image, let the shape of the CNN output be $1\times c\times h1\times w1$.

2. Suppose that selective search generates n region proposals. These region proposals (of different shapes) mark regions of interest (of different shapes) on the CNN output. Then these regions of interest further extract features of the same shape (say height h2 and width w2 are specified) in order to be easily concatenated. To achieve this, the fast R-CNN introduces the *region of interest (RoI) pooling* layer: the CNN output and region proposals are input into this layer, outputting concatenated features of shape n×c×h2×w2 that are further extracted for all the region proposals.
3. Using a fully connected layer, transform the concatenated features into an output of shape n×d, where d depends on the model design.
4. Predict the class and bounding box for each of the n region proposals. More concretely, in class and bounding box prediction, transform the fully connected layer output into an output of shape n×q (q is the number of classes) and an output of shape n×4, respectively. The class prediction uses softmax regression.

The region of interest pooling layer proposed in the fast R-CNN is different from the pooling layer introduced in CNN. In the pooling layer, we indirectly control the output shape by specifying sizes of the pooling window, padding, and stride. In contrast, we can directly specify the output shape in the region of interest pooling layer.

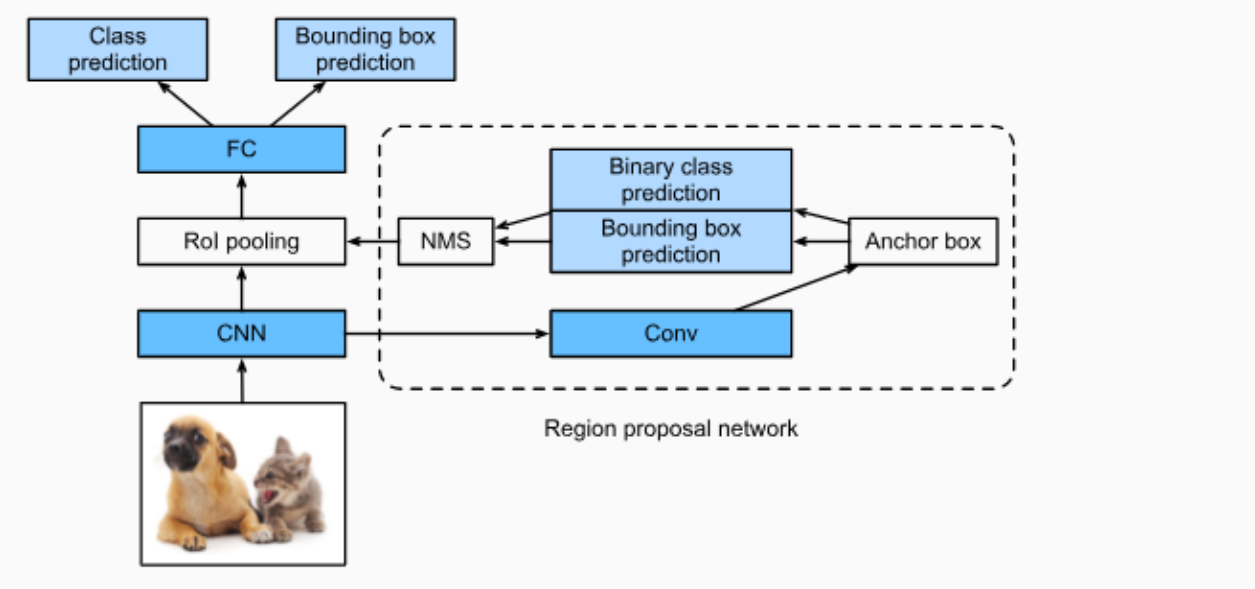
For example, let's specify the output height and width for each region as h2 and w2, respectively. For any region of interest window of shape h×w, this window is divided into a h2×w2 grid of subwindows, where the shape of each subwindow is approximately (h/h2)×(w/w2). In practice, the height and width of any subwindow shall be rounded up, and the largest element shall be used as output of the subwindow. Therefore, the region of interest pooling layer can extract features of the same shape even when regions of interest have different shapes.

As an illustrative example, in below figure , the upper-left 3×3 region of interest is selected on a 4×4 input. For this region of interest, we use a 2×2 region of interest pooling layer to obtain a 2×2 output. Note that each of the four divided subwindows contains elements 0, 1, 4, and 5 (5 is the maximum); 2 and 6 (6 is the maximum); 8 and 9 (9 is the maximum); and 10.



Faster R-CNN

To be more accurate in object detection, the fast R-CNN (2015) model usually has to generate a lot of region proposals in selective search. To reduce region proposals without loss of accuracy, the *faster R-CNN* proposes to replace selective search with a *region proposal network*.



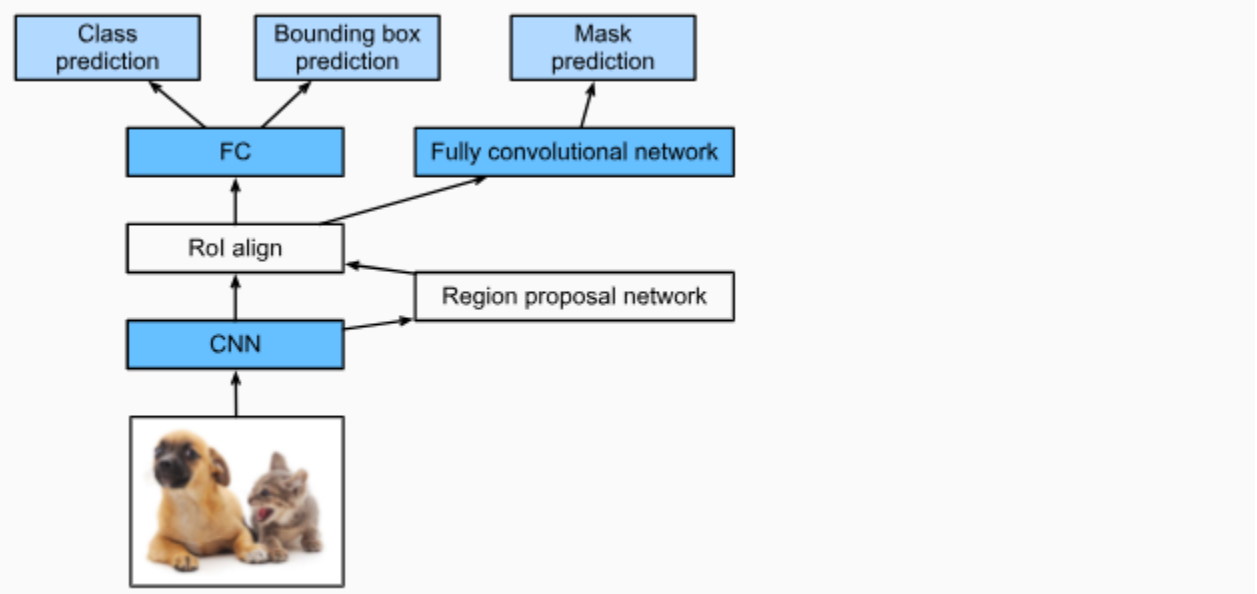
Above figure shows the faster R-CNN model. Compared with the fast R-CNN, the faster R-CNN only changes the region proposal method from selective search to a region proposal network. The rest of the model remain unchanged. The region proposal network works in the following steps:

1. Use a 3×3 convolutional layer with padding of 1 to transform the CNN output to a new output with c channels. In this way, each unit along the spatial dimensions of the CNN-extracted feature maps gets a new feature vector of length c.
2. Centered on each pixel of the feature maps, generate multiple anchor boxes of different scales and aspect ratios and label them.
3. Using the length-c feature vector at the center of each anchor box, predict the binary class (background or objects) and bounding box for this anchor box.
4. Consider those predicted bounding boxes whose predicted classes are objects. Remove overlapped results using non-maximum suppression. The remaining predicted bounding boxes for objects are the region proposals required by the region of interest pooling layer.

It is worth noting that, as part of the faster R-CNN model, the region proposal network is jointly trained with the rest of the model. In other words, the objective function of the faster R-CNN includes not only the class and bounding box prediction in object detection, but also the binary class and bounding box prediction of anchor boxes in the region proposal network. As a result of the end-to-end training, the region proposal network learns how to generate high-quality region proposals, so as to stay accurate in object detection with a reduced number of region proposals that are learned from data.

Mask R-CNN

In the training dataset, if pixel-level positions of object are also labeled on images, the *mask R-CNN* introduced in 2017, can effectively leverage such detailed labels to further improve the accuracy of object detection



As shown in Figure, the mask R-CNN is modified based on the faster R-CNN. Specifically, the mask R-CNN replaces the region of interest pooling layer with the *region of interest (RoI) alignment* layer. This region of interest alignment layer uses bilinear interpolation to preserve the spatial information on the feature maps, which is more suitable for pixel-level prediction. The output of this layer contains feature maps of the same shape for all the regions of interest. They are used to predict not only the class and bounding box for each region of interest, but also the pixel-level position of the object through an additional fully convolutional network. More details on using a fully convolutional network to predict pixel-level semantics of an image will be provided in subsequent sections of this chapter.

The significant changes introduced by Mask R-CNN from Faster R-CNN are:

1. Replacing ROIPool with ROIAlign to handle the problem of misalignments between the input feature map;
2. The region of interest (ROI) pooling grid, and;
3. The use of Feature Pyramid Network (FPN) that enhances the capabilities of Mask R-CNN by providing a multi-scale feature representation, enabling efficient feature reuse, and handling scale variations in objects.

What sets Mask R-CNN apart is its ability to not only detect objects within an image but also to precisely segment and identify the pixel-wise boundaries of each object. This fine-grained

segmentation capability is accomplished through the addition of an extra "mask head" branch to the Faster R-CNN model.

In this blog post, we will take an in-depth look at Mask R-CNN works, where the model performs well, and what limitations exist with the model.

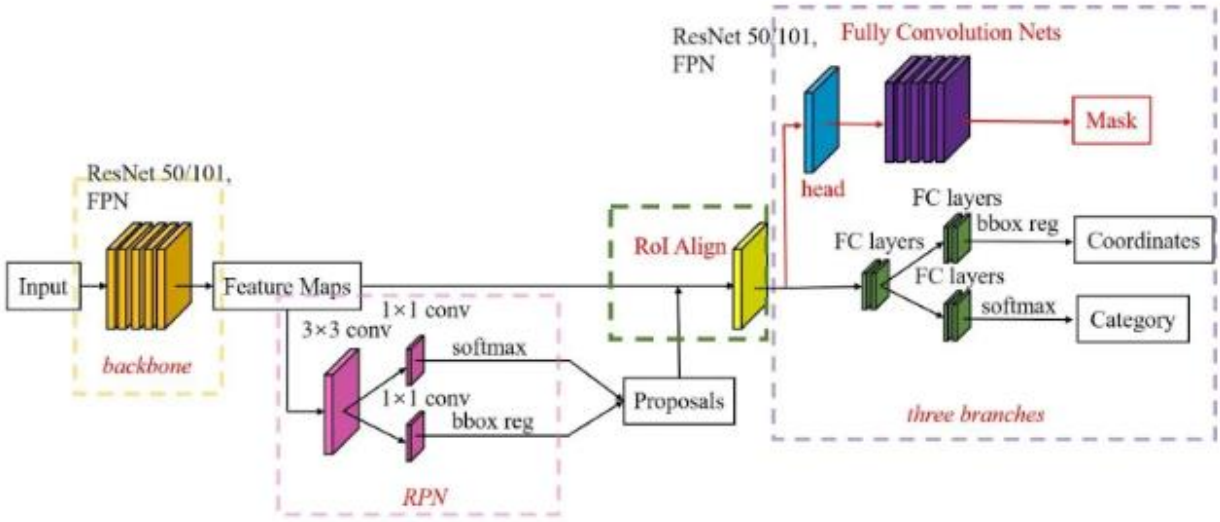
Without further ado, let's begin!

Mask R-CNN is a deep learning model that combines object detection and instance segmentation. It is an extension of the Faster R-CNN architecture.

The key innovation of Mask R-CNN lies in its ability to perform pixel-wise instance segmentation alongside object detection. This is achieved through the addition of an extra **"mask head"** branch, which generates precise segmentation masks for each detected object. This enables fine-grained pixel-level boundaries for accurate and detailed instance segmentation.

Two critical enhancements integrated into Mask R-CNN are ROIAlign and Feature Pyramid Network (FPN). ROIAlign addresses the limitations of the traditional ROI pooling method by using bilinear interpolation during the pooling process. This mitigates misalignment issues and ensures accurate spatial information capture from the input feature map, leading to improved segmentation accuracy, particularly for small objects.

FPN plays a pivotal role in feature extraction by constructing a multi-scale feature pyramid. This pyramid incorporates features from different scales, allowing the model to gain a more comprehensive understanding of object context and facilitating better object detection and segmentation across a wide range of object sizes.

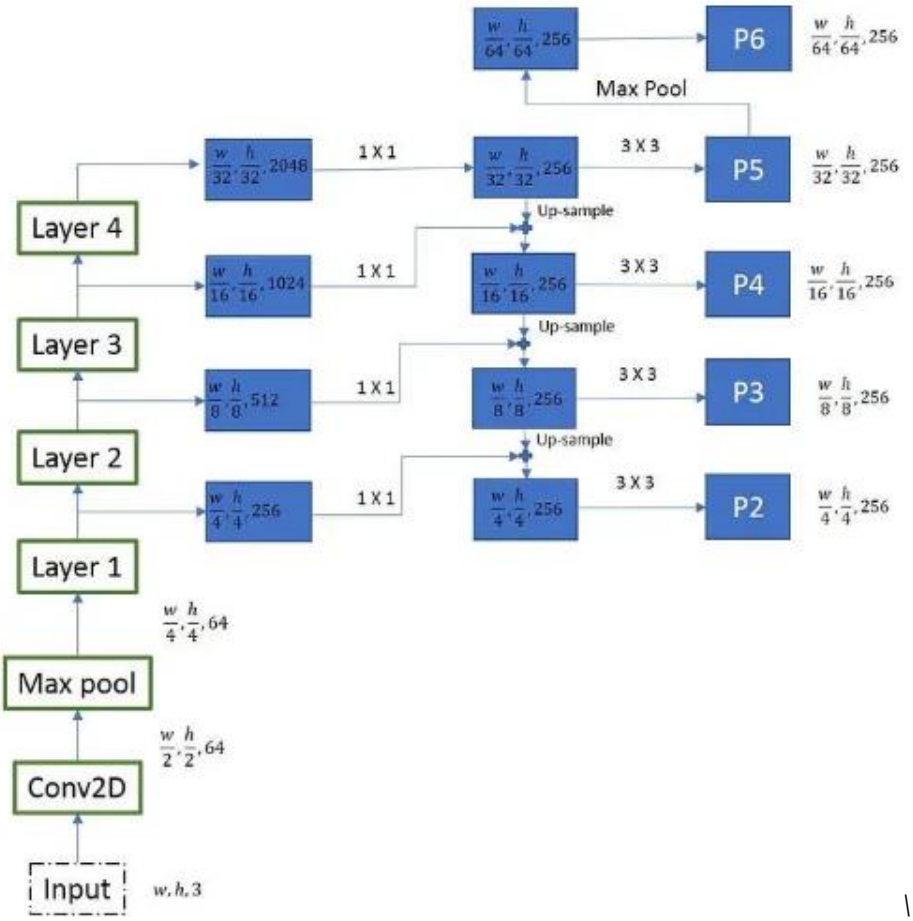


Mask R-CNN Architecture

The architecture of Mask R-CNN is built upon the Faster R-CNN architecture, with the addition of an extra "mask head" branch for pixel-wise segmentation. The overall architecture can be divided into several key components:

Backbone Network

The backbone network in Mask R-CNN is typically a pre-trained convolutional neural network, such as ResNet or ResNeXt. This backbone processes the input image and extracts high-level features. An FPN is then added on top of this backbone network to create a feature pyramid.
FPNs are designed to address the challenge of handling objects of varying sizes and scales in an image. The FPN architecture creates a multi-scale feature pyramid by combining features from different levels of the backbone network. This pyramid includes features with varying spatial resolutions, from high-resolution features with rich semantic information to low-resolution features with more precise spatial details.



The FPN in Mask R-CNN consists of the following steps:

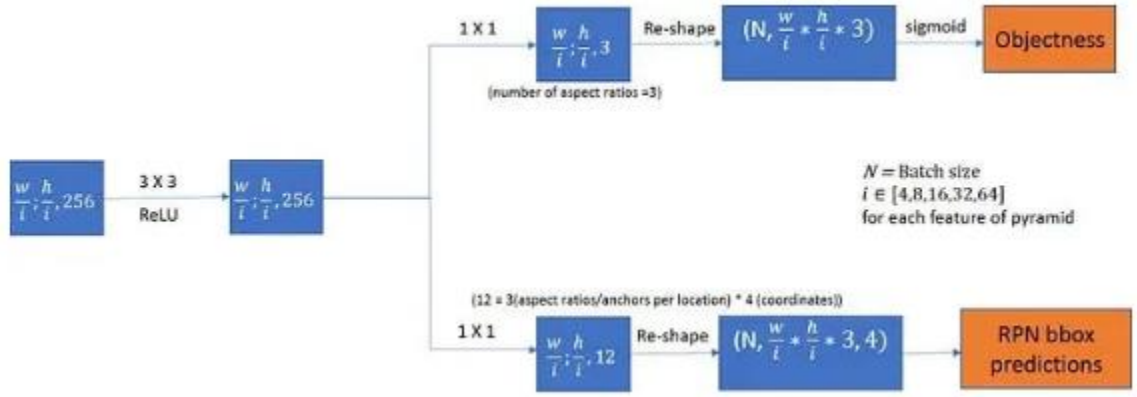
1. **Feature Extraction**: The backbone network extracts high-level features from the input image.
2. **Feature Fusion**: FPN creates connections between different levels of the backbone network to create a top-down pathway. This top-down pathway combines high-level semantic information with lower-level feature maps, allowing the model to reuse features at different scales.
3. **Feature Pyramid**: The fusion process generates a multi-scale feature pyramid, where each level of the pyramid corresponds to different resolutions of features. The top level of

the pyramid contains the highest-resolution features, while the bottom level contains the lowest-resolution features.

The feature pyramid generated by FPN enables Mask R-CNN to handle objects of various sizes effectively. This multi-scale representation allows the model to capture contextual information and accurately detect objects at different scales within the image.

Region Proposal Network (RPN)

The RPN is responsible for generating region proposals or candidate bounding boxes that might contain objects within the image. It operates on the feature map produced by the backbone network and proposes potential regions of interest.

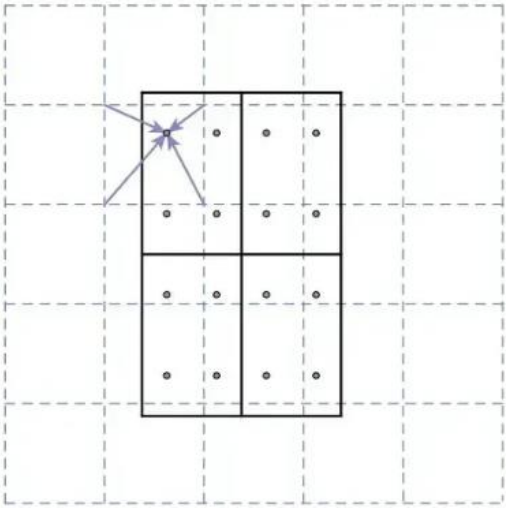


ROIAlign

After the RPN generates region proposals, the ROIAlign (Region of Interest Align) layer is introduced. This step helps to overcome the misalignment issue in ROI pooling.

ROIAlign plays a crucial role in accurately extracting features from the input feature map for each region proposal, ensuring precise pixel-wise segmentation in instance segmentation tasks.

The primary purpose of ROIAlign is to align the features within a region of interest (ROI) with the spatial grid of the output feature map. This alignment is crucial to prevent information loss that can occur when quantizing the ROI's spatial coordinates to the nearest integer (as done in ROI pooling).



The ROIAlign process involves the following steps:

1. **Input Feature Map**: The process begins with the input feature map, which is typically obtained from the backbone network. This feature map contains high-level semantic information about the entire image.
2. **Region Proposals**: The Region Proposal Network (RPN) generates region proposals (candidate bounding boxes) that might contain objects of interest within the image.
3. **Dividing into Grids**: Each region proposal is divided into a fixed number of equal-sized spatial bins or grids. These grids are used to extract features from the input feature map corresponding to the region of interest.
4. **Bilinear Interpolation**: Unlike ROI pooling, which quantizes the spatial coordinates of the grids to the nearest integer, ROIAlign uses bilinear interpolation to calculate the pooling contributions for each grid. This interpolation ensures a more precise alignment of the features within the ROI.
5. **Output Features**: The features obtained from the input feature map, aligned with each grid in the output feature map, are used as the representative features for each region proposal. These aligned features capture fine-grained spatial information, which is crucial for accurate segmentation.

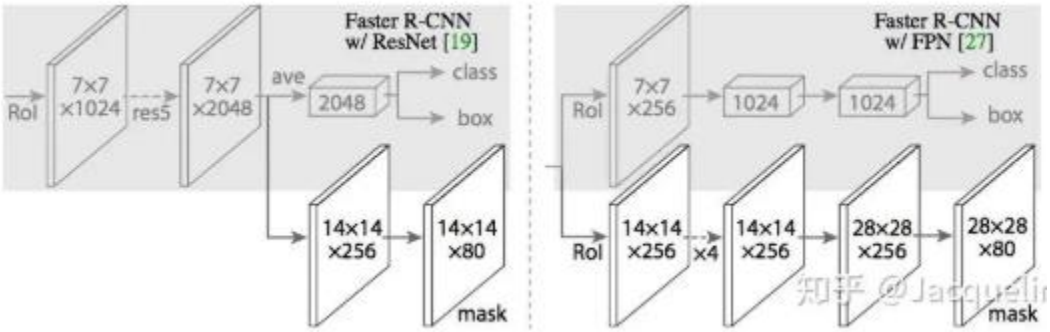
By using bilinear interpolation during the pooling process, ROIAlign significantly improves the accuracy of feature extraction for each region proposal, mitigating misalignment issues.

This precise alignment enables Mask R-CNN to generate more accurate segmentation masks, especially for small objects or regions that require fine details to be preserved. As a result, ROIAlign contributes to the strong performance of Mask R-CNN in instance segmentation tasks.

Mask Head

The Mask Head is an additional branch in Mask R-CNN, responsible for generating segmentation masks for each region proposal. The head uses the aligned features obtained through ROIAlign to predict a binary mask for each object, delineating the pixel-wise boundaries

of the instances. The Mask Head is typically composed of several convolutional layers followed by upsample layers (deconvolution or transposed convolution layers).



During training, the model is jointly optimized using a combination of classification loss, bounding box regression loss, and mask segmentation loss. This allows the model to learn to simultaneously detect objects, refine their bounding boxes, and produce precise segmentation masks.

Mask R-CNN Limitations

The COCO 2015 and 2016 segmentation challenges were won by the MNC and FCIS models, respectively. Surprisingly, Mask R-CNN achieved better results than the more intricate FCIS

Mask R-CNN excels in multiple areas, making it a powerful model for various computer vision tasks such as object detection, instance segmentation, multi-object segmentation, and complex scene handling.

However, Mask R-CNN has some limitations to consider:

- **Computational Complexity**: Training and inference can be computationally intensive, requiring substantial resources, especially for high-resolution images or large datasets.
- **Small-Object Segmentation**: Mask R-CNN may struggle to accurately segment very small objects due to limited pixel information.
- **Data Requirements**: Training Mask R-CNN effectively requires a large amount of annotated data, which can be time-consuming and expensive to acquire.
- **Limited Generalization to Unseen Categorie**s: The model's ability to generalize to unseen object categories is limited, especially when data is scarce.

Conclusion

Mask R-CNN merges object detection and instance segmentation, providing the capability to not only detect objects but also to precisely delineate their boundaries at the pixel level. By using a Feature Pyramid Network (FPN) and the Region of Interest Align (ROIAlign), Mask R-CNN achieves strong performance and accuracy.

Mask R-CNN does have certain limitations, such as its computational complexity and memory usage during training and inference. It may encounter challenges in accurately segmenting very small objects or handling heavily occluded scenes. Acquiring a substantial amount of annotated training data can also be demanding, and fine-tuning the model for specific domains may require careful parameter tuning.